

Physics-Inspired Machine Learning, Fall 2026

Homework 1

Due Date: September 21, 2026 at 11:59 PM

Both your code and written answers must be uploaded on Gradescope. No late homeworks will be accepted (unless you have late days remaining). Please make sure your Name, Student ID number, Date, and Assignment number are in the top-right hand corner of the page. Number all pages in the bottom-right hand corner. Please describe your thought process and state all assumptions made for each question when relevant. Each question is prefaced with either [code] if we expect to see code or [written] if you must provide a written answer. For questions marked [code + written], both are required.

Please assign pages to the relevant problems on Gradescope when submitting. This helps speed up grading so you get feedback faster!

Problem 1 - QM7 Molecules (50 points)

In this problem, we will consider QM7 [1, 3], a dataset of stable organic molecules with up to 23 atoms.

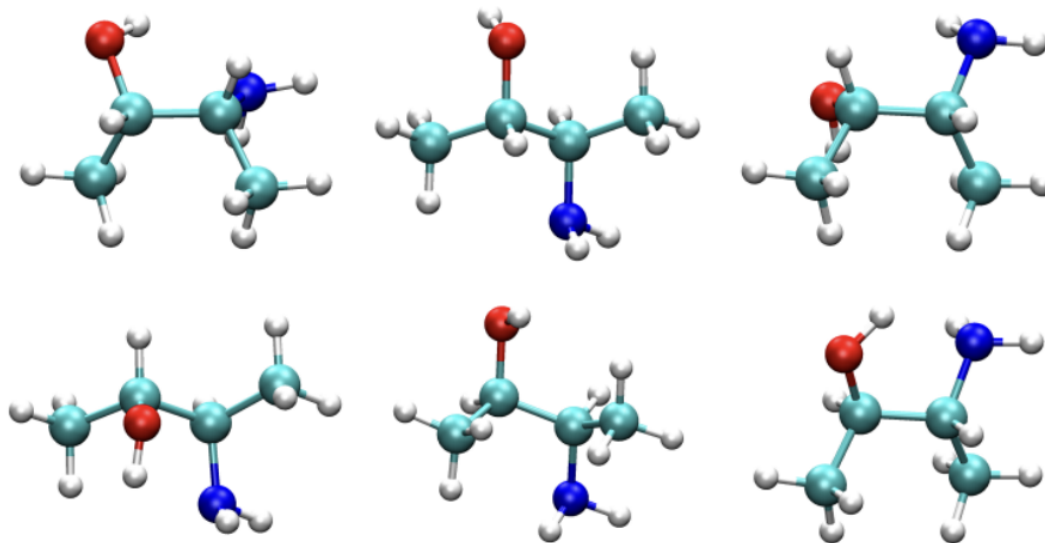


Figure 1: Example molecules from the QM-7 dataset [2].

The dataset will be automatically downloaded the first time you run the training script (`train.py`). The task we will consider is predicting the atomization energy of a molecule given the 3D coordinates and identities of its constituent atoms. This is a canonical task in computational chemistry, with applications including molecular dynamics simulations and drug-design.

We'll employ a very naive strategy for this task. Using the 3D atomic coordinates (field 'R' of the dataset, shape of $(23, 3)$) and atomic numbers (field 'Z' of the dataset, shape of (23)) as input features, we'll use a multilayer perceptron (MLP) to predict the atomization energy (a scalar) for each molecule. Formally, the goal is to learn a mapping $E_\theta : \mathbb{R}^{N \times 3} \times \mathbb{Z}^N \rightarrow \mathbb{R}$, where E_θ is a neural network with parameters

θ , and N is the number of atoms in the molecule. In the dataset, variable-size molecules are handled by zero-padding all matrices to have a first dimension of size 23. The MLP will consist of the following layers: 1) flattening and concatenation of the input coordinates and atomic numbers, 2) projection to 400 dimensions, 2) hyperbolic tangent activation, 3) projection to 100 dimensions, 4) hyperbolic tangent activation function, 5) projection to output dimension, 6) un-normalization using the mean and standard deviation of the training set atomization energies. We'll use a simple squared loss to train the MLP:

$$L(\theta) = \frac{1}{2} \sum_{i=1}^B (E_{\theta}(R_i, Z_i) - E_{i,\text{true}})^2,$$

where B is the batch size, and R_i, Z_i and $E_{i,\text{true}}$ are the 3D coordinates, atomic numbers, and ground truth energies of the i^{th} molecule in the batch. Note that we'll also be using two other related metrics, the root mean squared error (RMSE) and mean absolute error (MAE), to measure the performance of our models, but we won't be explicitly optimizing the models on these metrics.

We'll implement all neural network training using Numpy, so you won't need a GPU for this problem.

Using auto-differentiation libraries (e.g., TensorFlow, PyTorch, or JAX) on this problem is not allowed. All code for this problem can be found in the `problem1_qm7` folder. You will be modifying `model.py`, `train.py`, and `utils.py`.

- (i). (20 points) [code] Implement the forward and backward passes of the various components of the MLP by filling in the missing sections of code marked `TODO` in `model.py`. Hint: in the backwards pass functions, you'll want to set the gradients of the weights of that layer (i.e `self.DW = ...`), and return the gradient of the loss with respect to that layer's input.
- (ii). (10 points) [code + written] Implement the training loop by filling in the missing sections of code marked `TODO` in `train.py`
 - (a) Train your MLP using the command `python train.py`. The instructor solution took around 5 minutes to train. Include a plot of the RMSE during training, and report the final train and test RMSE/MAEs by running `python test.py`. If you experience training instabilities, you can try playing around with the learning rate.
- (iii). (3 points) [written] State-of-the-art ML methods can achieve a test MAE of less than 2 kcal/mol on this dataset. Provide some explanations as to why our current MLP-based approach does not achieve this performance.
- (iv). (5 points) [code + written] We will next try to improve our model by adding rotational data augmentations. This serves a "soft constraint" which encourages the model's energy predictions to be invariant to rotations. Implement data augmentation by writing the function `rotate_3d_coordinates` in `utils.py`. 3-D rotation matrices about the x, y, and z-axes are given by,

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{pmatrix}, R_y(\theta) = \begin{pmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{pmatrix}, R_z(\theta) = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

where the angle θ is given in radians.

- (v). (4 points) [code + written] Apply online data augmentations by calling `rotate_3d_coordinates` appropriately in `train.py`. The x, y, and z-axis rotation angles should be sampled uniformly between -180 and 180 degrees. Each minibatch element should be rotated independently. The ground truth energy of the rotated molecules is the same as that of the original molecules.
 - (a) Train your MLP with data augmentations using the command `python train.py --augmentation`. The instructor solution took around 7 minutes to train. Include a plot of the training RMSE and report the final train and test RMSE/MAEs by running `python test.py --augmentation`.

Explain any differences you observe relative to training without data augmentations.

- (vi). (3 points) [written] What are the limitations of data augmentation in this context?
- (vii). (5 points) [written] Propose an alternative method to represent the input data that would eliminate the need for rotational data augmentation while keeping the same neural network architecture. What are the remaining limitations with such an approach? Hint: consider a task where the prediction target is a vector (e.g per-atom forces) rather than a scalar.

Problem 2 - Viscous Burgers' Equation (50 points)

We'll be studying *Burgers' equation*, a 1D non-linear partial differential equation modeling transport phenomena in fluids. As noted in lecture, PDEs are traditionally solved with numerical solvers. Alternatively, one can use a neural network in lieu of a traditional solver; within the area of neural PDE solvers, Burgers' is a standard benchmark task. The system of equations describes the behaviour of u , the speed of viscous fluid over a space, x , and time, t . The Burgers' system is governed by the following equations:

$$\frac{\partial u(x, t)}{\partial t} + \frac{1}{2} \frac{\partial u^2(x, t)}{\partial x} = \nu \frac{\partial^2 u(x, t)}{\partial x^2} \quad x \in (0, 1), t \in (0, 1) \quad (1)$$

$$u(x, 0) = u_0(x) \quad x \in (0, 1) \quad (2)$$

$$u(0, t) = u(1, t) \text{ and } \frac{\partial u(0, t)}{\partial x} = \frac{\partial u(1, t)}{\partial x} \quad \text{Periodic Boundary Conditions} \quad (3)$$

where ν is the viscosity parameter and u_0 is a known initial condition. For our problem setting, we will focus on one viscosity, $\nu = 0.01$.

We want to solve for u using a neural network on an N_x (number of spatial points) by N_t (number of temporal points) regularly spaced grid. The neural network learns to map from a discretized spatio-temporal domain ($N_x \times N_t \times 2$) and initial condition (N_x dimensional vector) to a solution approximating $u(x, t) \in \mathbb{R}^{N_x \times N_t}$. The spatio-temporal domain is a matrix of $N_x \times N_t \times 2$, where each entry consists of an x and t value. Formally, the learning objective is to find $f_\theta : \mathbb{R}^{N_x \times N_t \times 2} \times \mathbb{R}^{N_x} \rightarrow \mathbb{R}^{N_x \times N_t}$ where θ parameterizes the neural network. For this assignment, you'll be implementing a naive 2D convolutional neural network.

Both the training and test set can be found in the `data` folder. The training and test sets contain 1000 and 50 unique initial conditions, respectively. All solutions are uniformly spaced over $(x, t) \in (0, 1)^2$ discretized to a resolution of 101 (N_x) $\times$ 101 (N_t). The code is located in `problem2.burgers` and code you will fill out is marked as `TODD`. This problem uses PyTorch. It will run faster on a GPU but can also run on a CPU.

- (i). (5 points) [code + written] Implement the dataset class for loading the initial condition, spatio-temporal grid, and full solution in `data.py`. We have provided the code required to read the Matlab files containing the training and test data. Recall our neural network maps from the spatio-temporal grid ($\mathbb{R}^{N_x \times N_t \times 2}$) and initial condition ($\mathbb{R}^{N_x}$) to $\mathbb{R}^{N_x \times N_t}$. To accommodate this, you should broadcast the initial condition across N_t and concatenate with `self.grid` to produce a model input of $N_x \times N_t \times 3$. Plot a few different training solution trajectories. What characteristics do you notice about the solutions?
- (ii). (3 points) [code] Next, fill out the 2D convolutional neural network class in `model.py`. You should

have three convolutional layers followed by two pointwise feature transformations. The convolutional layers must use a padding of 1, stride of 1, 64 feature maps, and kernel size of 3×3 . The pointwise features transformation layers should have a hidden dimension of 128. For all layers, except the final layer, use a tanh activation. The final layer should use no activation function. You may implement the pointwise feature transformation using either a fully connected layer or a 1×1 convolution.

- (iii). **Training Loop.** Set up all necessary pieces to train using a data loss.
- (2 points) [code] In `pde.py`, implement a L_2 relative loss in `burgers_data_loss`. If u represents the ground truth solution and $\hat{u}$ the neural network prediction, the L_2 relative loss is $\frac{\|\hat{u}-u\|_2}{\|u\|_2}$. We'll refer to this as the *data loss*.
 - (10 points) [code + written] Complete the training loop in `train.py` using a data loss. For this problem, you should use PyTorch's dataloader class to wrap the dataset from problem (i). Use Adam with a learning rate of $5e^{-3}$. Fix the number of epochs to 40 with a batch size of 16. The instructor solution took about 3 minutes to finish. Include your training loss plot. After training is complete, visualize the first 3 test instances predicted and note any qualitative differences between the ground truth and predicted solutions.
- (iv). **PDE Residual.** One method of improving neural PDE solvers is to leverage known physical properties about the system. Notably, for any valid solution to the Burgers' system, equation (1) must hold.
- (10 points) [written] Write the Taylor series expansion of $u(x, t)$ around $x + \Delta x$ and $x - \Delta x$. Using both series, derive the formula for a second-order **central** finite differences scheme to compute $\frac{\partial u(x, t)}{\partial x}$ and $\frac{\partial^2 u(x, t)}{\partial x^2}$. Show that the error for both approximations is $O(\Delta x^2)$. Hint: Central finite differences uses both $u(x + \Delta x, t)$ and $u(x - \Delta x, t)$ in the final formulas.
 - (5 points) [code] Equation (1) can be rewritten as:

$$\underbrace{\frac{\partial u(x, t)}{\partial t} + \frac{1}{2} \frac{\partial u^2(x, t)}{\partial x} - \nu \frac{\partial^2 u(x, t)}{\partial x^2}}_{\text{PDE Residual}} = 0 \quad (4)$$

the left-hand side of which we call the *PDE residual*. Using the above equations from part (a) for the first and second order derivatives, implement equation (4) in `burgers_pde_residual` in `pde.py`.

- (2 points) [written] Numerical differentiation is one way to compute derivatives. What is another method we could have used? Discuss some of the trade-offs and advantages.
 - (3 points) [code + written] We can compute a *PDE residual loss* by taking the mean squared error between the PDE residual and zero matrix (i.e., `torch.nn.MSELoss`). Evaluate the PDE residual loss over the entire training set and report the mean and standard deviation. Is this the value you expect? If not, discuss why this might be the case. *Note:* This does not require any training or the use of neural networks.
- (v). The PDE residual loss, when used during training, acts as a soft constraint. Using only the PDE residual loss, retrain the 2D convolutional network. Include your training loss plot.
- (2 points) [written] Report both the data loss and PDE residual loss over model predictions on the test set. Are the results what we expect? Discuss why or why not.
 - (3 points) [written] What loss terms (or soft constraints) can we add to improve the performance of our network? Hint: What other information does the problem formulation provide?
- (vi). **Constrained data loss.** Use both the PDE residual loss and data loss to train the same network. Weight the relative data loss with regularization weight $\alpha = 1$. For both parts, include your training loss plots.
- (2 points) [code + written] Report the PDE residual loss and data loss. How does this compare to using only the data loss?

- (b) (3 points) [code + written] Vary α in $(0, 1)$. Report the behaviour that you notice and which α leads to the lowest relative error. Hypothesize why this α achieves lower error than $\alpha = 1$.

References

- [1] L. C. Blum and J.-L. Reymond. “970 Million Druglike Small Molecules for Virtual Screening in the Chemical Universe Database GDB-13”. In: *J. Am. Chem. Soc.* 131 (2009), p. 8732.
- [2] J. Hoja, L. Medrano Sandonas, B.G. Ernst, et al. “QM7-X, a comprehensive dataset of quantum-mechanical properties spanning the chemical space of small organic molecules”. In: *Sci Data* 8 (2021), p. 43. DOI: 10.1038/s41597-021-00812-4.
- [3] M. Rupp et al. “Fast and accurate modeling of molecular atomization energies with machine learning”. In: *Physical Review Letters* 108 (2012), p. 058301.